Practical No.6

Aim:

To implement and demonstrate the Columnar Transposition Cipher, a permutation cipher that rearranges plaintext by writing it in rows and reading columns in the order specified by a numeric key.

Theory:

The Columnar Transposition Cipher is a cryptographic technique that:

- Writes plaintext in rows of a grid with width equal to the key length
- Reads columns in the order determined by a numeric key
- Concatenates column contents to form the ciphertext
- Preserves letter frequencies (only rearranges positions)
- Uses a numeric key where each digit indicates the reading order of columns

Key Features:

- Permutation-based cipher (no substitution)
- Key specifies column reading order (e.g., "4312567")
- Reversible process (decryption is inverse of encryption)
- Vulnerable to anagramming attacks

Algorithm (Encryption):

1. Calculate grid dimensions based on key length
2. Write plaintext in rows of the grid
3. Parse key to determine column reading order
4. Read each column in order specified by the key
5. Concatenate all column contents to form ciphertext

Algorithm (Decryption):

1. Calculate grid dimensions and column lengths
2. Parse key to determine column reading order
3. Distribute ciphertext across columns in key order
4. Read grid row by row to recover plaintext

Code:

```typescript
/**
 * Columnar Transposition Cipher Implementation
 *
 * A permutation cipher that rearranges plaintext by writing it in rows
 * and reading columns in the order specified by a numeric key.
 */

interface EncryptParameters {
  plainText: string
  key: string
}

interface DecryptParameters {
  cipherText: string
  key: string
}

/**
 * Parse numeric key into array of 0-indexed column positions
 * Key "4312567" means: read column in order 3, 2, 0, 1, 4, 5, 6
 * (col 1 is 4th, col 2 is 3rd, col 3 is 1st, col 4 is 2nd, etc.)
 */
function parseKey(key: string): number[] {
  const keyDigits = key.split('').map(Number)

  // Create array of column indices sorted by key value
  // Example: key=[4,3,1,2,5,6,7] -> order=[2,3,1,0,4,5,6]
  const order = keyDigits.map((_, index) => index)
  order.sort((a, b) => keyDigits[a] - keyDigits[b])

  return order
}

/**
 * Encrypt plaintext using Columnar Transposition cipher
 *
 * Algorithm:
 * 1. Write plaintext in rows of length = key length
 * 2. Read each column in order determined by key
 * 3. Concatenate columns to form ciphertext
 */
function columnarTranspositionEncrypt({
  plainText,
  key
}: EncryptParameters): string {
  const keyLength = key.length

  if (keyLength < 2) {
    throw new Error("Key must have at least 2 digits")
  }

  // Validate key is numeric
  if (!/^\d+$/.test(key)) {
    throw new Error("Key must contain only digits")
  }

  const numCols = keyLength
```

```
  const numRows = Math.ceil(plainText.length / numCols)

  // Create grid
  const grid: string[][] = []
  for (let row = 0; row < numRows; row++) {
    grid.push([])
    for (let col = 0; col < numCols; col++) {
      const index = row * numCols + col
      grid[row].push(index < plainText.length ? plainText[index] : '')
    }
  }

  // Parse key to get column reading order
  const colOrder = parseKey(key)

  // Read columns in order specified by key
  let ciphertext = ''
  for (const colIndex of colOrder) {
    for (let row = 0; row < numRows; row++) {
      if (grid[row][colIndex]) {
        ciphertext += grid[row][colIndex]
      }
    }
  }

  return ciphertext
}

/**
 * Decrypt ciphertext using Columnar Transposition cipher
 *
 * Algorithm:
 * 1. Calculate grid dimensions
 * 2. Determine column lengths (last column may be shorter)
 * 3. Distribute ciphertext across columns in key order
 * 4. Read row by row to recover plaintext
 */
function columnarTranspositionDecrypt({
  cipherText,
  key
}: DecryptParameters): string {
  const keyLength = key.length

  if (keyLength < 2) {
    throw new Error("Key must have at least 2 digits")
  }

  if (!/^\d+$/.test(key)) {
    throw new Error("Key must contain only digits")
  }

  const numCols = keyLength
  const numRows = Math.ceil(cipherText.length / numCols)

  // Calculate column lengths
  const colLengths: number[] = []
  for (let col = 0; col < numCols; col++) {
    const fullRows = Math.floor(cipherText.length / numCols)
```

```
    const extra = col < (cipherText.length % numCols) ? 1 : 0
    colLengths.push(fullRows + extra)
  }

  // Parse key to get column reading order
  const colOrder = parseKey(key)

  // Build grid by distributing ciphertext
  const grid: string[][] = Array.from({ length: numRows }, () =>
    new Array(numCols).fill('')
  )

  let textIndex = 0
  for (const colIndex of colOrder) {
    for (let row = 0; row < colLengths[colIndex]; row++) {
      grid[row][colIndex] = cipherText[textIndex++]
    }
  }

  // Read row by row
  let plaintext = ''
  for (let row = 0; row < numRows; row++) {
    for (let col = 0; col < numCols; col++) {
      if (grid[row][col]) {
        plaintext += grid[row][col]
      }
    }
  }

  return plaintext
}

// ============ TEST CASE ============

console.log('='.repeat(70))
console.log('COLUMNAR TRANSPOSITION CIPHER - TEST CASE')
console.log('='.repeat(70))

const KEY = '4312567'
const PLAINTEXT = 'attackpostponeduntiltwoam'

console.log(`\nKey: ${KEY}`)
console.log(`Plaintext: ${PLAINTEXT}`)
console.log(`Key length: ${KEY.length} columns`)

// Show the grid layout
const numCols = KEY.length
const numRows = Math.ceil(PLAINTEXT.length / numCols)

console.log(`\nGrid Layout (${numRows} rows x ${numCols} columns):`)
console.log('-'.repeat(70))

const grid: string[][] = []
for (let row = 0; row < numRows; row++) {
  const rowData: string[] = []
  for (let col = 0; col < numCols; col++) {
    const index = row * numCols + col
    rowData.push(index < PLAINTEXT.length ? PLAINTEXT[index] : ' ')
```

```
  }
  grid.push(rowData)
  console.log(`Row ${row + 1}: ${rowData.join('  ')}`)
}

// Show column reading order
const colOrder = parseKey(KEY)
console.log('\nColumn Reading Order (by key):')
console.log('-'.repeat(70))
for (let i = 0; i < colOrder.length; i++) {
  const col = colOrder[i]
  const colData = grid.map((row) => row[col]).join('')
  console.log(`Position ${i + 1}: Column ${col + 1} -> "${colData}"`)
}

const encrypted = columnarTranspositionEncrypt({ plainText: PLAINTEXT, key: KEY
    })
console.log('\n' + '-'.repeat(70))
console.log(`Ciphertext: ${encrypted}`)

const decrypted = columnarTranspositionDecrypt({
  cipherText: encrypted,
  key: KEY
})
console.log(`Decrypted: ${decrypted}`)

// Verification
console.log('\n' + '='.repeat(70))
console.log('VERIFICATION')
console.log('='.repeat(70))

if (decrypted === PLAINTEXT) {
  console.log('[OK] DECRYPTION VERIFIED: Successfully recovered original
    plaintext')
} else {
  console.log(`[FAIL] Decryption failed: expected '${PLAINTEXT}', got '${
    decrypted}'`)
}
```

Listing 1: Columnar Transposition Cipher Implementation

Output:

Conclusion:

The Columnar Transposition Cipher has been successfully implemented and demonstrated. The program correctly encrypts plaintext by arranging it in a grid and reading columns in the order specified by the numeric key. Decryption successfully recovers the original plaintext by reversing the process. The cipher preserves letter frequencies since it only rearranges character positions without substitution, making it vulnerable to frequency analysis and anagramming attacks. The implementation includes proper error handling for key validation and demonstrates both encryption and decryption processes with detailed grid visualization.

practical6-output.png

Figure 1: Output of Columnar Transposition Cipher Implementation